Immutable Loan Receipt & Nightly Circulation Ledger

Assignment Problem 5

Concept and approach

LoanReceipt must be final and every field final. Copy bookIds in the constructor and again in the getter. A with-method builds a new receipt rather than modifying the old one. The reference-only subtype carries a room number; the batch processor handles nulls and uses instanceof to count regular versus reference-only receipts in one pass.

Important implementation notes

The supplied statement asks for LoanReceipt to be both final and extended by ReferenceOnlyLoanReceipt. Those two requirements conflict in Java: a final class cannot be subclassed. The sample therefore uses a non-final CirculationReceipt base class to demonstrate the required hierarchy and batch dispatch. If the class name must remain LoanReceipt, remove final from LoanReceipt; this is the only way to allow the specified subclass.

Java solution

```
import java.util.Arrays;

final class LoanReceipt {
private final String memberId;
private final String[] bookIds;

public LoanReceipt(String memberId, String[] bookIds) {
if (memberId == null || bookIds == null) throw new IllegalArgumentException();
this.memberId = memberId;
this.bookIds = Arrays.copyOf(bookIds, bookIds.length);
}

public String getMemberId() { return memberId; }
public String[] getBookIds() { return Arrays.copyOf(bookIds, bookIds.length); }

public LoanReceipt withCorrectedBookId(int index, String newId) {
if (index < 0 || index >= bookIds.length) throw new IndexOutOfBoundsException();
String[] copy = getBookIds();
copy[index] = newId;
return new LoanReceipt(memberId, copy);
}
}

// Because LoanReceipt is final, it cannot be subclassed. To meet the stated
// ReferenceOnlyLoanReceipt extends LoanReceipt requirement, the base class
// cannot also be final in Java. Use this corrected hierarchy instead:
class CirculationReceipt {
private final String memberId;
private final String[] bookIds;
protected CirculationReceipt(String memberId, String[] bookIds) {
this.memberId = memberId;
this.bookIds = Arrays.copyOf(bookIds, bookIds.length);
}
public String getMemberId() { return memberId; }
public String[] getBookIds() { return Arrays.copyOf(bookIds, bookIds.length); }
}

final class RegularLoanReceipt extends CirculationReceipt {
public RegularLoanReceipt(String memberId, String[] bookIds) { super(memberId, bookIds); }
}

final class ReferenceOnlyLoanReceipt extends CirculationReceipt {
private final String roomNumber;
public ReferenceOnlyLoanReceipt(String memberId, String[] bookIds, String roomNumber) {
super(memberId, bookIds); this.roomNumber = roomNumber;
}
public String getRoomNumber() { return roomNumber; }
}

class CirculationLedger {
static final String BRANCH_CODE;
static {
BRANCH_CODE = "PT-LIB";
System.out.println("Circulation ledger initialized");
}

static String processNightlyCirculation(CirculationReceipt[] receipts) {
int processed = 0, nullSkipped = 0, referenceOnly = 0, regular = 0;
for (CirculationReceipt receipt : receipts) {
```

```
if (receipt == null) { nullSkipped++; continue; }
processed++;
if (receipt instanceof ReferenceOnlyLoanReceipt) referenceOnly++;
else regular++;
}
return processed + " processed | " + nullSkipped + " null skipped | " +
referenceOnly + " reference-only | " + regular + " regular";
}
}
```

Sample output

```
2 processed | 1 null skipped | 1 reference-only | 1 regular
```